



DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S

Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Exhaustive Search

- Exhaustive Search is a brute – force problem solving technique
- It suggests generating each and every element of the problem domain, selecting those of them that satisfy all the constraints and then finding a desired element
- The desired element might be one which minimizes or maximizes a certain characteristic
- Typically the problem domain involves combinatorial objects such as permutations, combinations and subsets of a given set

DESIGN AND ANALYSIS OF ALGORITHMS

Exhaustive Search - Method

- Generate a list of all potential solutions to the problem in a systematic manner
- Evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
- When search ends, announce the solution(s) found

DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

- Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?

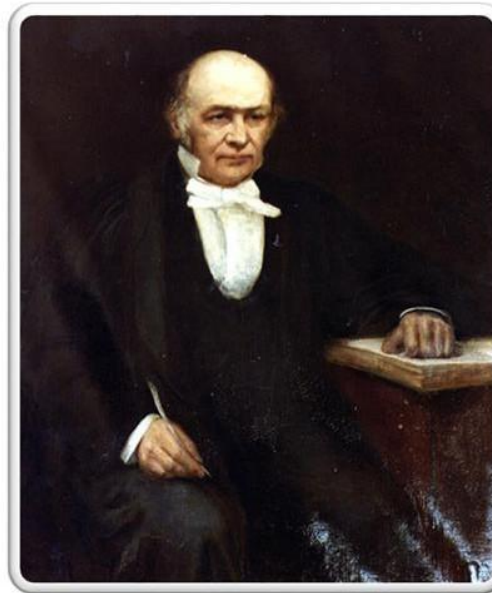
Alternative way to state the problem:

- Find the shortest Hamiltonian Circuit in a weighted connected graph

DESIGN AND ANALYSIS OF ALGORITHMS

TSP: History and Relevance

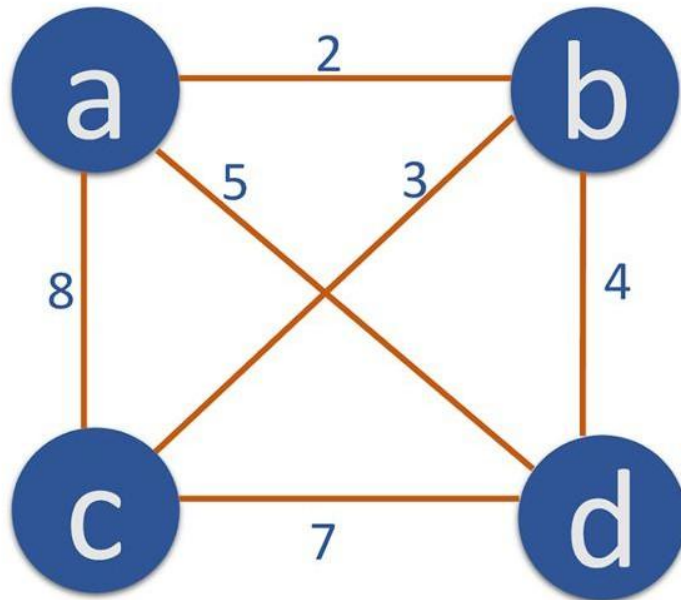
- The Travelling Salesman Problem was mathematically formulated by Irish Mathematician Sir William Rowan Hamilton
- It is one of the most intensively studied problems in optimization
- It has applications in logistics and planning



DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

Example



Tour	Length
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2+3+7+5 = 17$
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2+4+7+8 = 21$
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$8+3+4+5 = 20$
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$8+7+4+2 = 21$
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$5+4+3+8 = 20$
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$5+7+3+2 = 17$

DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

Efficiency

- The Exhaustive Search solution to the Travelling Salesman problem can be obtained by keeping the origin city constant and generating permutations of all the other $n - 1$ cities
- Thus, the total number of permutations needed will be $(n - 1)!$

Algorithm **Travelling Salesperson Problem**

```
mincost ← Infinity
```

```
for each permutation of (n - 1) cities
```

```
    cost ← 0
```

```
    for each edge in the Hamiltonian circuit
```

```
        cost ← cost + cost of the edge
```

```
    if (cost < mincost)
```

```
        mincost ← cost
```

```
return mincost
```

Input Size: n

Basic Operation : addition of cost of an edge

$C(n) = n * (n - 1)! = n! \in \Theta(n!)$

DESIGN AND ANALYSIS OF ALGORITHMS

Knapsack Problem

Given n items:

- weights: $w_1 \ w_2 \ \dots \ w_n$
- values: $v_1 \ v_2 \ \dots \ v_n$
- a knapsack of capacity W

Find the most valuable subset of items that fit into the knapsack

DESIGN AND ANALYSIS OF ALGORITHMS

Knapsack Problem

Example

Knapsack Capacity $W = 16$

Item	Weight	Value
1	2	20
2	5	30
3	10	50
4	5	10

DESIGN AND ANALYSIS OF ALGORITHMS

Knapsack Problem

Subset	Total Weight	Total Value
{1}	2	20
{2}	5	30
{3}	10	50
{4}	5	10
{1, 2}	7	50
{1, 3}	12	70
{1, 4}	7	30
{2, 3}	15	80
{2, 4}	10	40
{3, 4}	15	60
{1, 2, 3}	17	Not Feasible
{1, 2, 4}	12	60
{1, 3, 4}	17	Not Feasible
{2, 3, 4}	20	Not Feasible
{1, 2, 3, 4}	22	Not Feasible

Knapsack Problem by Exhaustive Search

DESIGN AND ANALYSIS OF ALGORITHMS

Knapsack Problem

- The Exhaustive Search solution to the Knapsack Problem is obtained by generating all subsets of the set of n items given and computing the total weight of each subset in order to identify the feasible subsets
- The number of subsets for a set of n elements is 2^n
- The Exhaustive Search solution to the Knapsack Problem belongs to $\Omega(2^n)$

DESIGN AND ANALYSIS OF ALGORITHMS

The Assignment Problem

- There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is $C[i, j]$. Find an assignment that minimizes the total cost

DESIGN AND ANALYSIS OF ALGORITHMS

The Assignment Problem

Example

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

DESIGN AND ANALYSIS OF ALGORITHMS

The Assignment Problem

Algorithmic Plan

1. Generate all legitimate assignments
2. Compute their costs
3. Select the cheapest one

DESIGN AND ANALYSIS OF ALGORITHMS

The Assignment Problem

The Assignment Problem by Exhaustive Search

Assignment	Cost
1, 2, 3, 4	$9 + 4 + 1 + 4 = 18$
1, 3, 4, 2	$9 + 8 + 9 + 7 = 33$
1, 4, 3, 2	$9 + 6 + 1 + 7 = 23$
1, 4, 2, 3	$9 + 6 + 3 + 8 = 26$
1, 3, 2, 4	$9 + 8 + 3 + 4 = 24$
1, 2, 4, 3	$9 + 4 + 9 + 8 = 30$

etc.,

DESIGN AND ANALYSIS OF ALGORITHMS

The Assignment Problem

Efficiency

- The Assignment Problem is solved by generating all permutations of n
- The number of permutations for a given number n is $n!$
- Therefore, the exhaustive search is impractical for all but very small instances of the problem

Algorithm **AssignmentProblem**($C[1..n, 1..n]$)

//Input: $n \times n$ cost matrix C .

//Output: Cost of the cheapest assignment.

mincost $\leftarrow$ Infinity

for each permutation of n jobs

cost $\leftarrow$ 0, **person** $\leftarrow$ 1

for each job in the assignment

cost $\leftarrow$ **cost** + $C[\text{person}, \text{job}]$

person $\leftarrow$ **person** + 1

if (**cost** < **mincost**) **mincost** = **cost**

return **mincost**

DESIGN AND ANALYSIS OF ALGORITHMS

Brute Force



Input Size: n , Basic Operation : cost + $C[\text{person}, \text{job}]$

$$C(n) = n * n! \in \Theta((n*n!))$$

Exhaustive search is often classified as a brute-force approach for solving combinatorial problems. Which of the following statements best captures the primary source of computational complexity in exhaustive search algorithms.

- a) The difficulty lies mainly in verifying whether a candidate solution satisfies the problem constraints.
- b) The complexity primarily arises from the need to generate all combinatorial objects in the problem domain, whose number grows exponentially with input size.
- c) Exhaustive search is inefficient because it lacks optimization techniques such as greedy or dynamic programming methods.
- d) The complexity is dominated by the objective function evaluation, which is usually polynomial in time.

Answer : **b)** The complexity primarily arises from the need to generate all combinatorial objects in the problem domain, whose number grows exponentially with input size.

Which of the following statements best justifies modeling the Travelling Salesperson Problem (TSP) as the problem of finding a shortest Hamiltonian circuit in a weighted graph?

- a) Because every weighted graph always contains a Hamiltonian circuit.
- b) Because visiting each city exactly once and returning to the start corresponds to a cycle passing through all vertices exactly once.
- c) Because Hamiltonian circuits minimize edge weights by definition.
- d) Because graph models eliminate the need for permutations of cities.

Answer b) Because visiting each city exactly once and returning to the start corresponds to a cycle passing through all vertices exactly once.

In the solution to the Travelling Salesman problem, Even after eliminating symmetric tours by considering only one direction (e.g., enforcing that city b precedes city c), the exhaustive-search approach remains impractical. Which of the following best explains why?

- a) The reduction only removes polynomially many cases, while the total number of tours is exponential.
- b) The number of tours remains proportional to $1/2(n-1)!$ which still grows factorially with n
- c) Directional symmetry elimination introduces additional constraint-checking overhead.
- d) The remaining tours may not include the optimal solution.

Answer **b)** The number of tours remains proportional to $1/2(n-1)!$ which still grows factorially with n

In Travelling Salesman problem, After fixing the starting vertex, the exhaustive-search approach generates all permutations of the remaining $n-1$ cities. Which of the following best characterizes the resulting time complexity of this approach?

- a) $O(n^2)$ since each tour requires computing its total length.
- b) $O((n-1)!)$, because all permutations of the intermediate cities must be examined.
- c) $O(2^{n-1})$, due to binary choices of including or excluding cities.
- d) $O(n!)$ because the starting vertex is still part of the permutations.

Answer b: $O((n-1)!)$, because all permutations of the intermediate cities must be examined.

Which of the following statements best explains why the exhaustive-search approach to the knapsack problem has time complexity $\Theta(2^n)$?

- a) Because each item can be either partially or fully included in the knapsack.
- b) Because all permutations of the items must be examined to ensure optimality.
- c) Because every subset of the n items must be generated and evaluated for feasibility and value.
- d) Because the knapsack capacity W grows exponentially with the number of items.

Answer c) Because every subset of the n items must be generated and evaluated for feasibility and value

why are exhaustive-search algorithms for the knapsack problems considered extremely inefficient on every input?

- a) They repeatedly evaluate infeasible solutions even after finding an optimal one.
- b) Their worst-case time complexity is exponential and cannot be reduced by efficient subset generation alone.
- c) They require excessive memory to store all candidate solutions simultaneously.
- d) Their inefficiency arises only for specially constructed worst-case inputs.

Answer C) Their worst-case time complexity is exponential and cannot be reduced by efficient subset generation alone

DESIGN AND ANALYSIS OF ALGORITHMS

Exhaustive Search : Multiple choice question



Which of the following conclusions most accurately reflects the significance of classifying the knapsack and traveling salesman problems as NP-hard?

- a) These problems cannot be solved exactly and must always use approximation algorithms
- b) Exhaustive search is the only possible algorithmic approach for such problems.
- c) No polynomial-time algorithm is currently known for these problems, and it is widely believed that none exists.
- d) Backtracking and branch-and-bound guarantee polynomial-time solutions for all instances.

Answer: C) No polynomial-time algorithm is currently known for these problems, and it is widely believed that none exists.

DESIGN AND ANALYSIS OF ALGORITHMS

Exhaustive Search : Multiple choice question



Which of the following statements best explains why the exhaustive-search approach to the assignment problem requires examining $n!$ candidate solutions?

- a) Because each person can be assigned to any subset of jobs.
- b) Because feasible assignments correspond one-to-one with permutations of the integers $1, 2, \dots, n$
- c) Because the cost matrix contains n^2 independent cost values.
- d) Because selecting the minimum cost in each row leads to conflicts

Answer b) Because feasible assignments correspond one-to-one with permutations of the integers $1, 2, \dots, n$

DESIGN AND ANALYSIS OF ALGORITHMS

Exhaustive Search : Multiple choice question



In Job Assignment problem Selecting the smallest element in each row does not necessarily lead to an optimal solution. What is the fundamental algorithmic reason for this failure?

- a) Greedy selection ignores the cumulative sum of costs across rows
- b) The smallest entries in different rows may belong to the same column, violating feasibility constraints.
- c) Greedy strategies increase the time complexity beyond factorial.
- d) The smallest element in the matrix always forces a suboptimal global solution.

Answer b) The smallest entries in different rows may belong to the same column, violating feasibility constraints.



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu